



Settlex

AI agents negotiate B2B invoices and B2C returns, a human approves with a Passkey, and the money settles on Stellar and lands in Turkish lira.

RISE IN × STELLAR PRO HACKATHON 2026
GENESIS TRACK · STELLAR TESTNET

PRO HACKATHON
19-20 SEPTEMBER, 2026

The money moves at the speed of email

B2B — invoice reconciliation

- Procurement receives an invoice above the agreed budget.
- Someone emails the supplier and haggles over a few rounds.
- A manager signs off inside an email thread.
- Finance wires the money *days* later.

B2C — customer refunds

- Support negotiates store credit versus cash by hand.
- Cash goes out *before* the parcel is even back.
- Refund fraud eats the margin.
- Cross-border crypto still has to become lira in a bank account.

Every step needs a person, the approvals live in inboxes, and no payment ends where the business actually needs it: **TRY** in a Turkish IBAN.

An autonomous payment agent with human guardrails

- 1 **AI agents negotiate.** A buyer agent and a seller agent (Gemini 2.5 Flash) bargain in a few rounds, bounded by the company's rules and budget and informed by past invoices (RAG).
- 2 **No money moves on its own.** An agreed purchase waits for a manager's approval (maker-checker); an agreed cash refund is held in escrow until the returned parcel is inspected.
- 3 **Approval happens with a fingerprint.** Every approval triggers a fresh Passkey (FaceID / TouchID) check that the backend verifies.
- 4 **Settlement is real.** The payment goes on-chain as USDC on Stellar and is cashed out to a Turkish IBAN in TRY through an anchor (SEP-1 / 10 / 12 / 38 / 6).
- 5 **The company funds the agent from its own wallet** — Freightier or any wallet via Stellar Wallets Kit.

Days of back-and-forth, settled in seconds

3

rounds, not days

Buyer and seller agents close the deal live over WebSocket while the team watches.

0

payments without a human

Every lira leaves only after a maker-checker approval and a fresh biometric check.

1

receipt per settlement

An on-chain Stellar tx hash plus the anchor's TRY payout reference, every time.

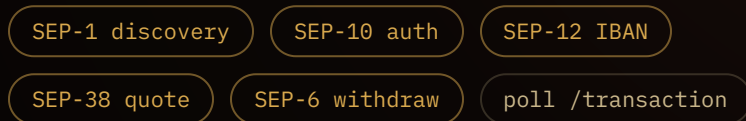
Features

- **AI-to-AI negotiation.** B2B price rounds inside the budget rule; B2C split refunds (part cash, part store credit) with the coupon rate computed by the system, not the LLM.
- **On-the-fly RAG.** Past invoices embedded with `text-embedding-004`, ranked by cosine similarity, top 3 injected into the buyer agent.
- **Rules engine.** Spending limits in natural language — “max 450 TL for cups from Kağıt Tedarik”.
- **Maker-checker purchases.** Agreed deals sit as `PENDING_APPROVAL` until a manager approves or rejects.
- **Escrow refunds.** Cash refunds wait as `PENDING_INSPECTION` until the parcel is inspected.
- **Biometric approval at signing time.** The backend accepts only a WebAuthn assertion from the last 60 seconds, usable once.
- **Fault-tolerant settlement.** `COMPLETED` only with a Stellar tx hash; otherwise rollback and a retryable `502`.
- **Bilingual & mobile-first.** UI and negotiation language switch between Turkish and English.

Invoice to lira, end to end



ANCHOR CALL CHAIN



PAYMENT STATE MACHINE

NEGOTIATING → PENDING_APPROVAL → AGREED → COMPLETED
 AGREED → PENDING_APPROVAL (rollback on failure)

Three layers, one settlement path

Browser

React 19 · Vite · Tailwind 4

Agent console, approvals and wallet cards. Stellar Wallets Kit (Freighter, xBull, Albedo, Lobstr). WebAuthn Passkey. Native WebSocket for live turns.

FastAPI backend

Python 3.11 · Pydantic 2

Negotiation service, return agent, rules engine, approvals with rollback, wallet funding (build → verify → submit), anchor client, Passkey auth, SQLite, WebSocket bus.

External

The rails

Gemini 2.5 Flash + text-embedding-004. Stellar Horizon testnet: payments and classic DEX path payments. TR TRY ⇌ USDC anchor at `tr-mock-anchor.fly.dev`.

COMPONENT

FILE

RESPONSIBILITY

Negotiation service

`agent_negotiation.py`

3-step buyer/seller loop, RAG, anomaly detection, holds agreed purchases

Approvals

`approvals.py`

Approve → settle (COMPLETED only with a tx hash) or roll back; reject

The AI never touches the treasury

RISK

CONTROL

Someone triggers agents or payments without permission

WebAuthn Passkey login with user verification. 256-bit sessions in SQLite, 12-hour expiry, required on every money path.

A stolen session approves a payment

Step-up approval: only a token from an assertion made in the last **60 seconds**, deleted on first use. A login session alone is not enough.

The AI moves money on its own

Agents produce an *agreement* only. Prices are clamped in code: at or below budget, never under 90% of the invoice, anomalies frozen for the CFO.

A failed payment shows as paid, or is paid twice

COMPLETED only with a tx hash; failures roll back to pending. Status leaves *pending* before the first await, so a double click gets **409**.

A tampered wallet transaction

The signed XDR is decoded and rejected unless it is signed, from the connected wallet, and a single payment to the agent.

Honest scope: testnet only, no role separation yet, open Passkey registration, session in **localStorage**, no rate limiting — all listed in the

Verified on Stellar testnet

WHAT	RESULT	TRANSACTION
Purchase approved → TRY payout	150 TL deal → SEP-38 quote → 3.0901616 USDC sent → anchor paid 150.00 TRY (FAST-SZJ8PP42A8)	0603ce54...
Off-ramp	1 USDC → 48.54 TRY via FAST (FAST-CM5SG8HJG1)	52b06cdd...
DEX top-up	XLM → USDC path payment for the agent wallet	b183e4d5...
Company funds the agent	25 XLM signed by the company wallet, verified and submitted by the backend	e5cca875...

AGENT WALLET

GC07KVCZRALTIDDX7AXPLVHTOV72JVS0...CPBM

ANCHOR TREASURY

GCLCZEQZ2THTEDAOFI66LACNPLY40BKN...T3Z6

Every settlement is stored with its `tx_hash` and linked to Stellar Expert in the console.

Requirements at a glance

REQUIREMENT	HOW SETTLEX MEETS IT	STATUS
1. Integration eligible partner	Stellar Wallets Kit connects Freightier, xBull, Albedo, Lobstr. The connected wallet signs the SEP-10 login and the budget transfer to the agent.	DONE
2. Anchor local payments	TRY $\rightleftharpoons$ USDC anchor <code>tr-mock-anchor.fly.dev</code> : SEP-1, SEP-10, SEP-12, SEP-38, SEP-6 and payout polling.	DONE
3. Core feature	Settlement <i>is</i> the product: every approved purchase and escrowed refund ends in a Stellar payment and a TRY payout.	DONE
Deployed on testnet	All payments run on Stellar testnet via Horizon testnet.	DONE
Passkeys nice-to-have	WebAuthn login plus a step-up biometric check before every payment approval.	DONE

Built with

BACKEND Python 3.11+, FastAPI 0.115, Pydantic 2, Unicorn

AI Google Gemini 2.5 Flash · text-embedding-004 + NumPy (RAG)

STELLAR stellar-sdk 12, Horizon testnet, classic DEX path payments

ANCHOR SEP-1, SEP-10, SEP-12, SEP-38, SEP-6 over httpx

WALLETS @creit.tech/stellar-wallets-kit 2.6

AUTH WebAuthn Passkeys (webauthn 2.5)

STORAGE SQLite + SQLAlchemy 2

FRONTEND React 19, Vite 8, Tailwind CSS 4, native WebSocket

STELLAR SKILLS REFERENCED DURING DEVELOPMENT

Anchors – SEP off-ramp flow

Frontend & Wallets – Wallets Kit

SEPs, CAPs & Ecosystem

Agentic Payments

Other tools: Stellar Python SDK · Horizon testnet · Friendbot · Stellar Expert · Stellar Lab · TR Mock Anchor.

Challenges we solved

- **The anchor could not be discovered.** The testnet USDC issuer's home domain publishes no transfer server, so discovery fell back to an anchor with no TRY. We pinned the TRY anchor with `ANCHOR_HOME`.
- **Amounts were off by ~50×**. The agreed TRY figure was sent as the same number of USDC. We now quote TRY→USDC with SEP-38 and send 7-decimal amounts.
- **The payout went to the wrong IBAN.** The anchor pays the IBAN on the SEP-12 customer, not the SEP-6 destination — so we register it before each withdrawal.
- **Failures were marked as paid.** Errors were swallowed and simulated payments still showed as settled. The approval path now rolls back and returns a retryable `502`.

DESIGN DECISIONS

- **Hold first, pay later** — keeps AI mistakes and prompt injection away from the treasury.
- **Server-verified step-up** — a browser FaceID prompt alone can be bypassed.
- **Negotiate in TRY, settle in USDC** — the business thinks in lira; SEP-38 locks the rate.
- **The wallet never gives up its key** — backend builds, wallet signs, backend verifies.

From testnet prototype to mainnet

WHEN	MILESTONE
Q4 2026	Role-based maker-checker (initiator ≠ approver), persistent approval queue, per-company agent wallets
Q4 2026	Move from the TRY testnet sandbox to a licensed Turkish anchor on mainnet — same SEP code, new <code>ANCHOR_HOME</code>
Q1 2027	Escrow and approval policy as a Soroban contract, so funds are locked on-chain rather than by the backend
Q1 2027	Apply to the Stellar Community Fund (SCF) / InstaAwards
Q2 2027	ERP / e-Fatura integrations and pilot customers, then mainnet launch

What “done” looks like: a company connects its ERP, sets its rules, and every invoice is negotiated, approved with a fingerprint and paid in TRY — with an on-chain audit trail behind it.

The Team

Musa Ok

Lead Backend & AI Agent Developer — negotiation service, approvals, anchor client, Passkeys.

Şahin Kara

Technical Documentation & Architecture — system design, SEP flows, submission docs.

Delil Çiya Avcı

Product Strategy & Presentation — positioning, demo narrative, go-to-market.

ACKNOWLEDGMENTS

Rise In and the Stellar Development Foundation for the hackathon, the TR Mock Anchor team for the TRY $\rightleftharpoons$ USDC testnet rail, Creit Tech for Stellar Wallets Kit, and Google for Gemini.



Thank you

Negotiated by agents. Approved by a fingerprint. Settled on Stellar.
Paid in lira.

[GITHUB.COM/MUSA-OK/SETTLEX-STELLAR-HACKATHON](https://github.com/MUSA-OK/SETTLEX-STELLAR-HACKATHON)

MIT · STELLAR TESTNET

RISE IN × STELLAR

PRO HACKATHON 2026